System Architecture & Design Document: Multi-Agent Research Factory

Technical Specification of 5-Agent Coordination, Pub/Sub Bus, and Epistemic Feedback Loops

Course: F0003 CA2/CA3 · Agentic AI Course | Track: Multi-Agent AI System

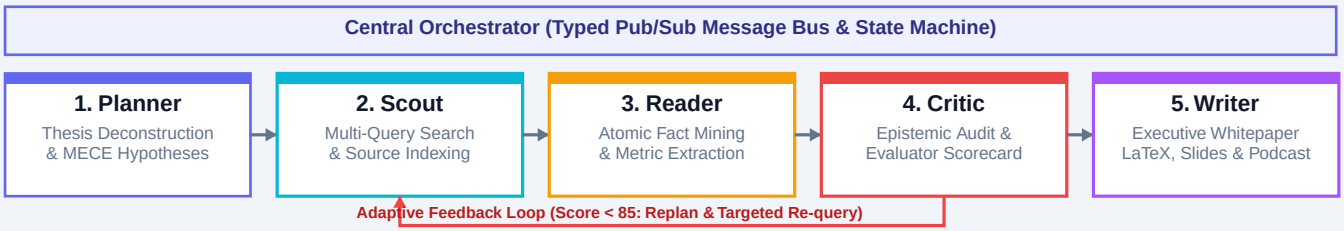
Team Members & PRNs:

- Anushka Tandon (23070122262) — Multi-Agent Architecture & Orchestrator
- Jyoti Kumari Sah (23070122278) — Epistemic Verification & Critic Engine
- Mustkarin Shah (23070122279) — Literature Scout & Retrieval Tools
- Emmanuel Lenge Wa Mpitshi (23070122260) — Reader Agent & Data Pipeline

1. Architectural Overview & Orchestration Pattern

The platform implements a hybrid Orchestrator-Evaluator-Optimizer architecture. A central ResearchOrchestrator manages system state and coordinates agent lifecycles over an asynchronous Event Bus. Rather than executing a rigid cascade, the orchestrator facilitates bidirectional negotiation between specialized worker agents and the adversarial Critic. Each agent is modeled with distinct system directives, specialized tool suites, and memory isolation.

FIGURE 1: 5-AGENT ARCHITECTURE & TYPED MESSAGE BUS TOPOLOGY



2. Agent Roles and Core Responsibilities

- Agent 1: Strategic Planner (PlannerAgent) — Analyzes user thesis, determines target audience, defines hypotheses, and outputs structured research vectors.
- Agent 2: Literature Scout (SearcherAgent) — Operates WebSearchTool and AcademicSearchTool, querying global search APIs, arXiv, and encyclopedic repositories.
- Agent 3: Deep Reader (ReaderAgent) — Implements DocumentChunkingTool and FactExtractionTool to distill high-density factual claims.
- Agent 4: Epistemic Critic (CriticAgent) — Audits facts against hallucination matrices, executes adversarial red-team stress tests, and triggers ReplanDirectives.
- Agent 5: Executive Synthesizer (WriterAgent) — Authors structured whitepapers with executive summaries, comparative tables, LaTeX math, and podcast briefing scripts.

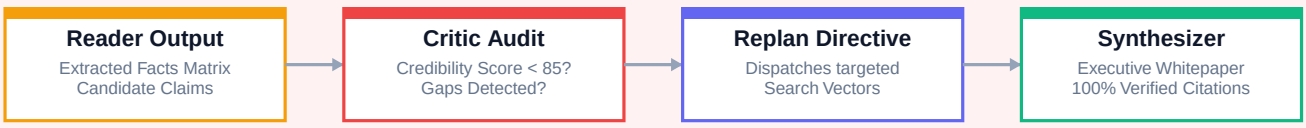
3. Inter-Agent Communication & Pub/Sub Message Bus

All agents communicate via a typed, audited in-memory Message Bus. Every message carries a unique ID, sender agent ID, recipient agent ID, ISO timestamp, message type, payload, and causal parentMessageId. This architecture guarantees full auditability, deterministic execution tracing, and enables real-time visual inspection in the Web UI.

4. The Non-Linear Critic Feedback Loop & Evaluator-Optimizer Sequence

If CriticAgent review produces an overallCredibilityScore < 85/100 or discovers unsupported claims, it emits an AGENT_MESSAGE with type "critic_rejection". The orchestrator increments the replanLoop counter, updates the Planner with targeted gaps, and re-dispatches the Scout to retrieve corroborating references. Once satisfied (or max loops reached), synthesis proceeds to the Writer.

FIGURE 2: NON-LINEAR EVALUATOR-OPTIMIZER FEEDBACK LOOP SEQUENCE



5. Technology Stack & Framework Justification

• Frontend: Next.js 14 App Router, React 18, Tailwind CSS, Lucide Icons with custom Liquid Glass, Claymorphism, Synthwave, and Cyberpunk design tokens.
• Multi-Agent Core: TypeScript, custom asynchronous pub/sub event bus, isolated tool harnesses.
• Model Integration: Google Gemini 1.5 Flash / Pro via @google/genai with fallback simulation engine.
• Testing & Verification: Vitest automated test suite with 6 test files covering tool execution, message bus, orchestrator, and feedback loop mechanics.